ASSIGNMENT 3

Combined I2C & SPI Protocol Implementation for Sensor Integration on STM32

DIYguru Education and Research Pvt. Ltd | Electric Vehicle Course | Nilabja Jana | 1st July Batch | 25/02/2026

3.1 Introduction

Modern electric vehicles rely on a diverse array of sensors to monitor the vehicle's physical state — motion, orientation, altitude, cabin temperature, and ambient pressure. This assignment integrates the MPU6050 (I2C, address 0x68) and BMP280 (SPI, CS on PA4) on STM32. I2C is chosen for the MPU6050 because it requires only two wires (SDA/SCL) and supports address-based multi-device selection. SPI is chosen for the BMP280 because it offers full-duplex, high-speed data transfer ideal for continuous pressure sampling. Both protocols run in parallel, demonstrating real-world automotive sensor fusion where motion data and environmental data are acquired and processed every second.

3.2 Objective

To integrate two different serial communication protocols simultaneously on STM32: I2C for the MPU6050 6-axis motion sensor (reading both accelerometer and gyroscope data) and SPI for the BMP280 environmental sensor (reading both temperature and pressure data). The system processes and converts all raw ADC values to meaningful engineering units, logs all six sensor channels via UART, and demonstrates multi-protocol automotive sensor fusion applicable to vehicle stability control, inertial navigation, and environmental monitoring. This version corrects three errors: a function-name typo, two missing operators in the Bosch compensation formulae, and replaces blocking HAL_MAX_DELAY timeouts with safe 50 ms values.

3.3 I2C vs SPI Protocol Comparison

Parameter	I2C (MPU6050)	SPI (BMP280)
Wires	2 — SDA, SCL	4 — MOSI, MISO, SCLK, CS
Speed	Standard 100 kHz / Fast 400 kHz	Up to 10 MHz (full-duplex)
Device Select	7-bit address (0x68/0x69)	Dedicated CS line per device
Duplex	Half-duplex (shared bus)	Full-duplex (simultaneous TX+RX)
Multi-device	Up to 127 devices on one bus	One CS pin per device
Best For	Config registers, low-speed sensors	High-speed, continuous data sensors

3.4 Sensor Theory

MPU6050 — 6-Axis IMU (I2C)

The MPU6050 combines a 3-axis MEMS accelerometer and 3-axis MEMS gyroscope. Each axis outputs a 16-bit signed integer. At default sensitivity (±2g), 1g = 16384 LSB; at ±250°/s, 1°/s = 131 LSB. Key registers: PWR_MGMT_1 (0x6B) — wake by writing 0x00; ACCEL_XOUT_H (0x3B)–0x40 — 6 bytes of accelerometer data; GYRO_XOUT_H (0x43)–0x48 — 6 bytes of gyroscope data.

BMP280 — Barometric Pressure & Temperature Sensor (SPI)

The BMP280 measures barometric pressure (300–1100 hPa) and temperature (-40 to +85°C) over SPI with the read bit set by OR-ing the register address with 0x80. Raw 20-bit ADC values must be converted using the Bosch compensation formula referencing 12 factory-trimmed calibration coefficients stored in registers 0x88–0x9F. Raw values are dimensionless and have no physical meaning without this compensation.

3.5 Pin Configuration

Pin	Peripheral	Protocol	Description
PB6	SCL	I2C1	I2C clock line to MPU6050
PB7	SDA	I2C1	I2C data line to MPU6050
PA4	BMP280_CS	SPI1 (GPIO)	Chip Select: LOW = selected, HIGH = idle
PA5	SPI1_SCK	SPI1	SPI clock to BMP280
PA6	SPI1_MISO	SPI1	Master In Slave Out — BMP280 data to STM32
PA7	SPI1_MOSI	SPI1	Master Out Slave In — STM32 commands to BMP280
PA9	USART1_TX	UART	115200 baud serial data logging

3.6 Code — Driver Files & main.c

mpu6050.h — I2C Driver Header

```
#ifndef MPU6050_H
#define MPU6050_H
#include "stm32f1xx_hal.h"

#define MPU6050_ADDR (0x68 << 1) /* 7-bit addr shifted for HAL */
#define MPU6050_PWR_MGMT 0x6B /* Power management register */ #define
MPU6050_ACCEL_REG 0x3B /* Accel X high byte start */ #define
MPU6050_GYRO_REG 0x43 /* Gyro X high byte start */

#define ACCEL_SCALE 16384.0f /* LSB/g at +-2g sensitivity */ #define
GYRO_SCALE 131.0f /* LSB/dps at +-250 dps */

#define I2C_TIMEOUT 50 /* FIX 3: 50 ms timeout (ms) */
#define SPI TIMEOUT 50 /* FIX 3: 50 ms timeout (ms) */

void MPU6050_Init (I2C_HandleTypeDef *hi2c);
void MPU6050_Read_Accel(I2C_HandleTypeDef *hi2c,
                        int16_t *ax, int16_t *ay, int16_t *az);
void MPU6050_Read_Gyro (I2C_HandleTypeDef *hi2c,
                        int16_t *gx, int16_t *gy, int16_t *gz);
#endif /* MPU6050_H */
```

Mpu6050.c — I2C Driver Implementation

```
#include "mpu6050.h"

void MPU6050_Init(I2C_HandleTypeDef *hi2c)
{
    uint8_t data = 0x00;
    /* FIX 3: I2C_TIMEOUT (50 ms) instead of HAL_MAX_DELAY */
    HAL_I2C_Mem_Write(hi2c, MPU6050_ADDR, MPU6050_PWR_MGMT,
                      1, &data, 1, I2C_TIMEOUT);
    HAL_Delay(100);
```

```
 void MPU6050_Read_Accel(I2C_HandleTypeDef *hi2c,
                         int16_t *ax, int16_t *ay, int16_t *az)
{
    uint8_t buf[6];
    /* FIX 3: I2C_TIMEOUT instead of HAL_MAX_DELAY */
    if (HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, MPU6050_ACCEL_REG,
                         1, buf, 6, I2C_TIMEOUT) != HAL_OK)
    {
        *ax = *ay = *az = 0; /* Safe default on timeout */ return;
    }
    *ax = (int16_t)((buf[0] << 8) | buf[1]);
    *ay = (int16_t)((buf[2] << 8) | buf[3]);
    *az = (int16_t)((buf[4] << 8) | buf[5]);
}

void MPU6050_Read_Gyro(I2C_HandleTypeDef *hi2c,
                       int16_t *gx, int16_t *gy, int16_t *gz)
{
    uint8_t buf[6];
    /* FIX 3: I2C_TIMEOUT instead of HAL_MAX_DELAY */
    if (HAL_I2C_Mem_Read(hi2c, MPU6050_ADDR, MPU6050_GYRO_REG,
                         1, buf, 6, I2C_TIMEOUT) != HAL_OK)
    {
        *gx = *gy = *gz = 0; /* Safe default on timeout */ return;
    }
    *gx = (int16_t)((buf[0] << 8) | buf[1]);
    *gy = (int16_t)((buf[2] << 8) | buf[3]);
    *gz = (int16_t)((buf[4] << 8) | buf[5]);
}
```

bmp280.h — SPI Driver Header

```
#ifndef BMP280_H
#define BMP280_H
#include "stm32f1xx_hal.h"

#define BMP280_CS_PORT GPIOA
#define BMP280_CS_PIN    GPIO_PIN_4

#define BMP280_READ_REG(r) ((r) | 0x80) /* Set bit7 for SPI read */

#define   BMP280_REG_CTRL   0xF4
#define   BMP280_REG_TEMP   0xFA
#define  BMP280_REG_PRESS  0xF7
#define BMP280_REG_CALIB 0x88

typedef struct {
    uint16_t  dig_T1;  int16_t  dig_T2;  int16_t  dig_T3;
    uint16_t  dig_P1;  int16_t  dig_P2;  int16_t  dig_P3;
    int16_t  dig_P4;  int16_t  dig_P5;  int16_t  dig_P6;
    int16_t dig_P7; int16_t dig_P8; int16_t dig_P9;
} BMP280_Calib_t;

void     BMP280_Init             (SPI_HandleTypeDef *hspi);
void     BMP280_Read_Calib       (SPI_HandleTypeDef *hspi, BMP280_Calib_t *cal);
int32_t BMP280_Read_RawTemp      (SPI_HandleTypeDef *hspi);


int32_t BMP280_Read_RawPressure(SPI_HandleTypeDef *hspi);      /* FIX 1: was MP280_ */ float
         BMP280_Compensate_Temp      (int32_t raw_t, BMP280_Calib_t *cal, int32_t *t_fine);
float    BMP280_Compensate_Pressure(int32_t raw_p, BMP280_Calib_t *cal, int32_t t_fine);
#endif /* BMP280_H */
```

bmp280.c — SPI Driver Implementation

```
/* ORIGINAL BUG — missing leading 'B': */
int32_t MP280_Read_RawPressure(...)      /* <-- compile error: undefined reference */

/* FIXED: */
int32_t BMP280_Read_RawPressure(...)     /* <-- correct name matches header + main.c */


/* BUG 1 — missing subtraction operator in pressure compensation: */ p
= 1048576 raw_p;         /* <-- syntax error, will not compile */
/* FIXED: */
p = 1048576 - raw_p;     /* <-- correct: raw ADC subtracted from 2^20 */

/* BUG 2 — missing assignment operator in temperature compensation: */
*t_fine var1 + var2;     /* <-- syntax error, will not compile */
/* FIXED: */
*t_fine = var1 + var2; /* <-- correct: result stored in t_fine for pressure calc */


#include "bmp280.h"

#define CS_LOW() HAL_GPIO_WritePin(BMP280_CS_PORT, BMP280_CS_PIN, GPIO_PIN_RESET) #define
CS_HIGH() HAL_GPIO_WritePin(BMP280_CS_PORT, BMP280_CS_PIN, GPIO_PIN_SET)

void BMP280_Init(SPI_HandleTypeDef *hspi)
{
    uint8_t cmd[2] = {BMP280_REG_CTRL, 0x27};
    CS_LOW();
    HAL_SPI_Transmit(hspi, cmd, 2, SPI_TIMEOUT); /* FIX 3 */
    CS_HIGH();
    HAL_Delay(10);
}

void BMP280_Read_Calib(SPI_HandleTypeDef *hspi, BMP280_Calib_t *cal)
{
    uint8_t reg = BMP280_READ_REG(BMP280_REG_CALIB);
    uint8_t  buf[24];
    CS_LOW();
    HAL_SPI_Transmit(hspi,  &reg,  1,  SPI_TIMEOUT);  /*  FIX  3  */
    HAL_SPI_Receive  (hspi,  buf,  24,  SPI_TIMEOUT);  /*  FIX  3  */
    CS_HIGH();
     cal->dig_T1  = (uint16_t)(buf[1]<<8|buf[0]);
     cal->dig_T2  = (int16_t)  (buf[3]<<8|buf[2]);
     cal->dig_T3  = (int16_t)  (buf[5]<<8|buf[4]);
     cal->dig_P1  = (uint16_t)(buf[7]<<8|buf[6]);
     cal->dig_P2  = (int16_t)  (buf[9]<<8|buf[8]);
     cal->dig_P3  = (int16_t)  (buf[11]<<8|buf[10]);
     cal->dig_P4  = (int16_t)  (buf[13]<<8|buf[12]);
     cal->dig_P5  = (int16_t)  (buf[15]<<8|buf[14]);
     cal->dig_P6  = (int16_t)  (buf[17]<<8|buf[16]);
     cal->dig_P7  = (int16_t)  (buf[19]<<8|buf[18]);
     cal->dig_P8  = (int16_t)  (buf[21]<<8|buf[20]);
     cal->dig_P9  = (int16_t)  (buf[23]<<8|buf[22]);
}

int32_t BMP280_Read_RawTemp(SPI_HandleTypeDef *hspi)
{
    uint8_t reg = BMP280_READ_REG(BMP280_REG_TEMP);
```

```
    uint8_t  buf[3];
    CS_LOW();
    HAL_SPI_Transmit(hspi,  &reg,  1,  SPI_TIMEOUT);  /*  FIX  3  */
    HAL_SPI_Receive  (hspi,  buf,  3,  SPI_TIMEOUT);  /*  FIX  3  */
    CS_HIGH();
    return (int32_t)((buf[0]<<12) | (buf[1]<<4) | (buf[2]>>4));
}

/* FIX 1: Corrected name — was MP280_Read_RawPressure (missing 'B') */ int32_t
BMP280_Read_RawPressure(SPI_HandleTypeDef *hspi)
{
    uint8_t reg = BMP280_READ_REG(BMP280_REG_PRESS);
    uint8_t  buf[3];
    CS_LOW();
    HAL_SPI_Transmit(hspi,  &reg,  1,  SPI_TIMEOUT);  /*  FIX  3  */
    HAL_SPI_Receive  (hspi,  buf,  3,  SPI_TIMEOUT);  /*  FIX  3  */
    CS_HIGH();
    return (int32_t)((buf[0]<<12) | (buf[1]<<4) | (buf[2]>>4));
}

/* FIX 2a: Restored missing '=' assignment operator */
float BMP280_Compensate_Temp(int32_t raw_t,
                              BMP280_Calib_t *cal, int32_t *t_fine)
{
    int32_t var1, var2;
    var1 = ((((raw_t>>3) - ((int32_t)cal->dig_T1<<1))) *
             ((int32_t)cal->dig_T2)) >> 11;
    var2 = (((((raw_t>>4) - ((int32_t)cal->dig_T1)) * ((raw_t>>4)
              - ((int32_t)cal->dig_T1))) >> 12) *
             ((int32_t)cal->dig_T3)) >> 14;
    *t_fine = var1 + var2; /* FIX 2a: was "*t_fine var1 + var2;" */
    return (float)((*t_fine * 5 + 128) >> 8) / 100.0f;
}

/* FIX 2b: Restored missing '-' subtraction operator */
float BMP280_Compensate_Pressure(int32_t raw_p,
                                  BMP280_Calib_t *cal, int32_t t_fine)
{
    int64_t var1, var2, p;
    var1 = ((int64_t)t_fine) - 128000;
    var2 = var1 * var1 * (int64_t)cal->dig_P6;
    var2 = var2 + ((var1 * (int64_t)cal->dig_P5) << 17);
    var2 = var2 + (((int64_t)cal->dig_P4) << 35);
    var1 = ((var1 * var1 * (int64_t)cal->dig_P3) >> 8) +
           ((var1 * (int64_t)cal->dig_P2) << 12);
    var1 = (((((int64_t)1) << 47) + var1)) *
           ((int64_t)cal->dig_P1) >> 33;
    if (var1 == 0) return 0.0f;
    p = 1048576 - raw_p; /* FIX 2b: was "p = 1048576 raw_p;" */ p
    = (((p << 31) - var2) * 3125) / var1;
    var1 = (((int64_t)cal->dig_P9) * (p>>13) * (p>>13)) >> 25;
    var2 = (((int64_t)cal->dig_P8) * p) >> 19;
    p = ((p + var1 + var2) >> 8) + (((int64_t)cal->dig_P7) << 4);
    return (float)p / 25600.0f;
}
```

main.c — Complete Integration

```
/* ORIGINAL (compile error): */
raw_press = MP280_Read_RawPressure(&hspi1);     /* undefined reference */

/* FIXED: */
raw_press = BMP280_Read_RawPressure(&hspi1); /* matches corrected function name */ #include

"main.h"
```

```
#include "mpu6050.h"
#include "bmp280.h"
#include <stdio.h>
#include <string.h>

I2C_HandleTypeDef hi2c1;
SPI_HandleTypeDef hspi1;
UART_HandleTypeDef huart1;

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_SPI1_Init();
    MX_USART1_UART_Init();

    MPU6050_Init(&hi2c1);
    BMP280_Init(&hspi1);

    BMP280_Calib_t bmp_cal;
    BMP280_Read_Calib(&hspi1, &bmp_cal);

    int16_t ax, ay, az, gx, gy, gz; float
    accel_x, accel_y, accel_z; float
    gyro_x, gyro_y, gyro_z; int32_t
    raw_temp, raw_press, t_fine; float
    temperature, pressure;
    char msg[220];

    while (1)
    {
        /* Step 1: Read MPU6050 Accelerometer */
        MPU6050_Read_Accel(&hi2c1, &ax, &ay, &az);
        accel_x = (float)ax / ACCEL_SCALE;
        accel_y = (float)ay / ACCEL_SCALE;
        accel_z = (float)az / ACCEL_SCALE;

        /* Step 2: Read MPU6050 Gyroscope */
        MPU6050_Read_Gyro(&hi2c1, &gx, &gy, &gz);
        gyro_x = (float)gx / GYRO_SCALE;
        gyro_y = (float)gy / GYRO_SCALE;
        gyro_z = (float)gz / GYRO_SCALE;

        /* Step 3: Read BMP280 Temperature */
        raw_temp     = BMP280_Read_RawTemp(&hspi1);
        temperature = BMP280_Compensate_Temp(raw_temp, &bmp_cal, &t_fine);

        /* Step 4: Read BMP280 Pressure — FIX 1: correct function name */ raw_press
        = BMP280_Read_RawPressure(&hspi1);
        pressure = BMP280_Compensate_Pressure(raw_press, &bmp_cal, t_fine);

        /* Step 5: Log all 8 channels via UART */ sprintf(msg,
        "ACCEL: X=%.2fg Y=%.2fg Z=%.2fg | ",
                accel_x, accel_y, accel_z);
        sprintf(msg + strlen(msg), "GYRO: X=%.1f Y=%.1f Z=%.1f deg/s | ",
                gyro_x, gyro_y, gyro_z);
        sprintf(msg + strlen(msg), "TEMP:%.2fC PRESS:%.2fhPa\r\n",
                temperature, pressure);
        HAL_UART_Transmit(&huart1,(uint8_t*)msg,strlen(msg),HAL_MAX_DELAY);

        HAL_Delay(1000);
    }
}
```

3.7 Results — UART Output (Docklight Serial Monitor)

```
/* Case 1: At rest on flat surface */
ACCEL: X=0.01g Y=-0.01g Z=1.00g | GYRO: X=0.1 Y=0.0 Z=-0.1 deg/s | TEMP:26.24C PRESS:1013.52hPa

/* Case 2: X-axis tilt (~30 deg forward) */
ACCEL: X=0.50g Y=-0.01g Z=0.87g | GYRO: X=12.5 Y=0.3 Z=-0.2 deg/s | TEMP:26.25C PRESS:1013.51hPa

/* Case 3: Y-axis tilt (~30 deg sideways) */
ACCEL: X=0.01g Y=-0.49g Z=0.87g | GYRO: X=0.2 Y=-11.8 Z=0.1 deg/s | TEMP:26.25C PRESS:1013.53hPa

/* Case 4: Dynamic shake (large multi-axis motion) */
ACCEL: X=0.87g Y=-0.62g Z=0.34g | GYRO: X=109.3 Y=-78.2 Z=54.2 deg/s | TEMP:26.27C PRESS:1013.50hPa
ACCEL: X=-0.73g Y=0.54g Z=0.43g | GYRO: X=-92.0 Y=68.1 Z=-43.3 deg/s | TEMP:26.27C PRESS:1013.50hPa

/* Case 5: BMP280 heat applied */
ACCEL: X=0.01g Y=-0.01g Z=1.00g | GYRO: X=0.1 Y=0.0 Z=-0.1 deg/s | TEMP:34.81C PRESS:1013.48hPa

/* Case 6: BMP280 cooled */
ACCEL: X=0.01g Y=-0.01g Z=1.00g | GYRO: X=0.1 Y=0.0 Z=-0.1 deg/s | TEMP:18.63C PRESS:1013.55hPa

/* Case 7: I2C bus fault — MPU6050 disconnected; FIX 3 returns after 50 ms */
ACCEL: X=0.00g Y=0.00g Z=0.00g | GYRO: X=0.0 Y=0.0 Z=0.0 deg/s | TEMP:26.24C PRESS:1013.52hPa
/* System continues normally — BMP280 unaffected. Was: system hung forever. */
```

Results Summary Table

Case	Accel X/Y/Z (g)	Gyro X/Y/Z (°/s)	Temp °C	Press hPa	State
Flat Rest	0.01/-0.01/1.00	0.1/0.0/-0.1	26.24	1013.52	Normal
X-Tilt	0.50/-0.01/0.87	12.5/0.3/-0.2	26.25	1013.51	Accel+Gyro X
Y-Tilt	0.01/-0.49/0.87	0.2/-11.8/0.1	26.25	1013.53	Accel+Gyro Y
Shake	Large variation	Large variation	26.27	1013.50	All axes OK
Heat	~0g rest	~0 deg/s	34.81	1013.48	BMP280 OK
Cooled	~0g rest	~0 deg/s	18.63	1013.55	BMP280 OK
I2C Fault	0.00/0.00/0.00	0.0/0.0/0.0	26.24	1013.52	Timeout 50ms

3.8 Conclusion

The integration of I2C and SPI protocols on a single STM32 platform proves the capability of the system to handle simultaneous, multi-protocol sensor fusion. By interfacing the MPU6050 over I2C and the BMP280 over SPI, the project successfully captures a full suite of motion and environmental data, including 3-axis acceleration, angular velocity, atmospheric pressure, and temperature. The transformation of raw ADC integers into meaningful engineering units (such as g-force, deg/s, and hPa) via scaling factors and complex Bosch compensation algorithms demonstrates a professional approach to embedded data processing. This unified sensing architecture provides the real-time data necessary for advanced vehicle stability control and environmental monitoring in electric vehicle applications.